

System Architecture Document - LLMOps Analytics Agents Platform

Version: 2.4 Date: 2026-06-23 Author: DangNghia Audience: Mentor / Evaluator

This document describes the LLMOps Analytics Agents platform architecture, about an **Agents Feature** inside Open WebUI where users ask NYC Taxi analytics questions, agents select the right table and SQL, DuckDB queries the S3-backed analytics DB, and the WebUI returns a business summary plus optional table or chart artifacts.

1. Problems and Solution Approach

Core Problems

Problem	Why it matters
Users need analytics without writing SQL	Business and operations users should ask questions in chat instead of opening a DB tool.
A natural-language question must be mapped to the correct table before SQL can be generated	With multiple tables in the registry, an ambiguous question can match several candidates — picking the wrong table produces wrong SQL and wrong results.
The model must not directly execute SQL or access arbitrary tables	Without a guardrail layer, a model-generated query could read unintended tables, run DDL, or chain statements — deterministic validation must sit between the model and the database.
Analytics data must be cheap and easy to refresh	The NYC Taxi dataset is analytical and file-oriented, so S3 Parquet is simpler than operating a dedicated OLAP database at this stage.
The multi-agent pipeline has multiple failure points that must be debuggable end-to-end	A request can fail at intent classification, table selection, SQL generation, validation, DuckDB execution, or summarization — each step needs to be individually traceable.
Locking application code to a specific model provider creates migration risk	Cost increases, latency degradation, or a stronger model requirement could force a provider swap — tight coupling would mean rewriting every agent call site.

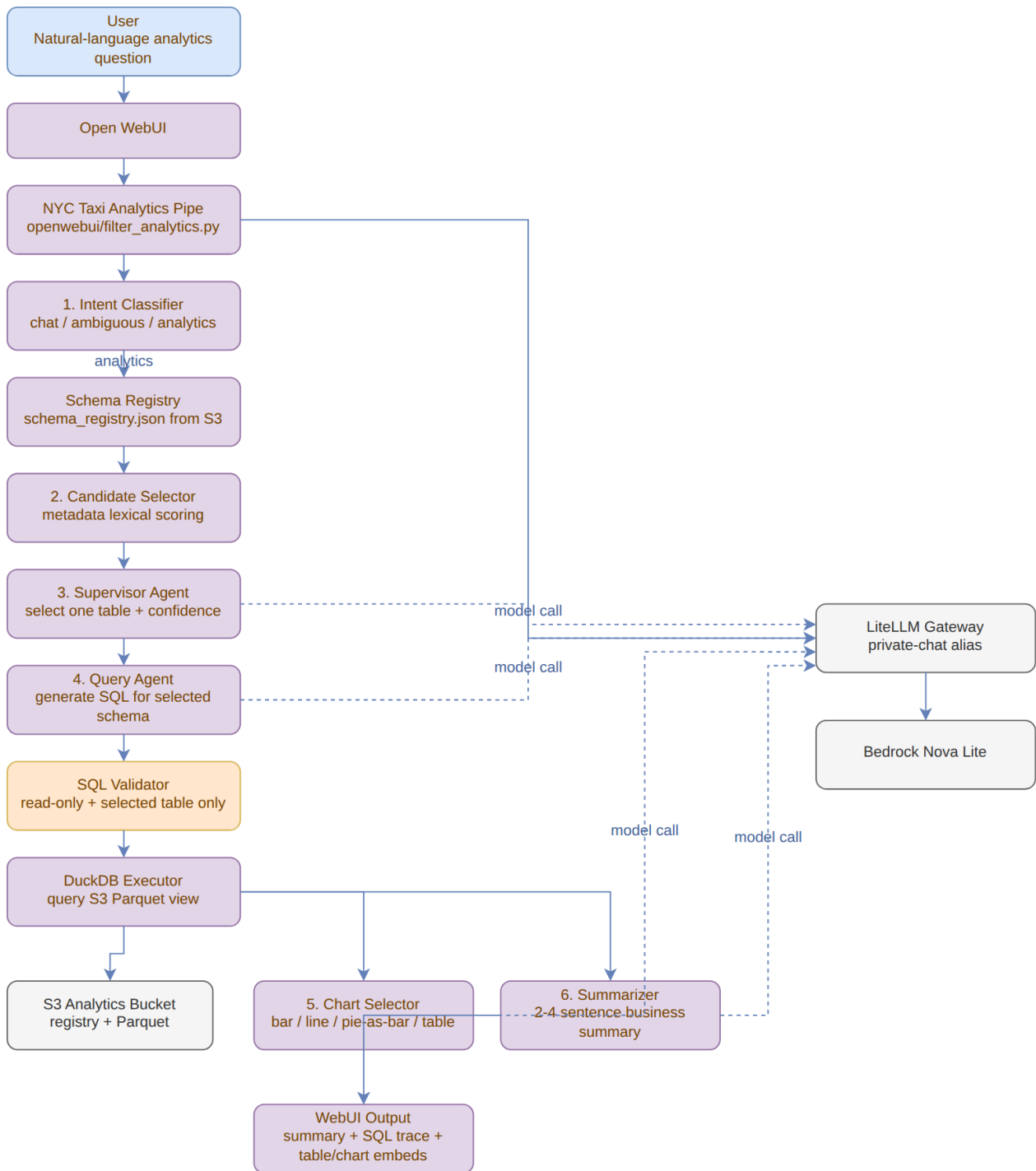
How the System Solves Them

Solution	Implementation in this repo
Chat-native analytics	Users ask in the same chat interface they already use — no separate tool, no SQL client. <code>openwebui/filter_analytics.py</code> intercepts analytics-intent messages and runs the pipeline.
Table disambiguation via metadata scoring	<code>_select_table_candidates()</code> scores each registry table by name, aliases, columns, and examples before any model call. The Supervisor Agent resolves ambiguity only when the lexical score is inconclusive.
Narrow agent roles	Intent classifier, supervisor, query agent, chart selector, and summarizer each handle one part of the workflow — isolating failures to a single step.
Guardrails before DB execution	The SQL validator blocks DDL, chained statements, file functions, unknown tables, and cross-table access before DuckDB executes anything.
S3-backed analytics DB	<code>schema_registry.json</code> and one Parquet prefix per table live in the analytics S3 bucket. Appropriate at current scale; concurrency and freshness limits are acknowledged tradeoffs (see Section 10).
Lightweight query runtime	DuckDB creates a view over the selected S3 table path and runs only validated SQL.
Provider abstraction via LiteLLM	Agents call LiteLLM through the <code>private-chat</code> alias; swapping Bedrock for another provider requires no change in the pipe or agent code.
End-to-end observability	Every request carries a <code>correlation_id</code> that threads through structured pipeline logs, Langfuse model traces, and Loki application logs — making it possible to replay any failed request step by step. Prometheus covers infra health; alerts fire to Slack within 5 minutes.

2. Agents Feature Overview

The agents feature is the main product surface in the current project. The user sees a chat interface, but the backend is a controlled multi-step pipeline rather than one model with free access to data.

Agents Architecture



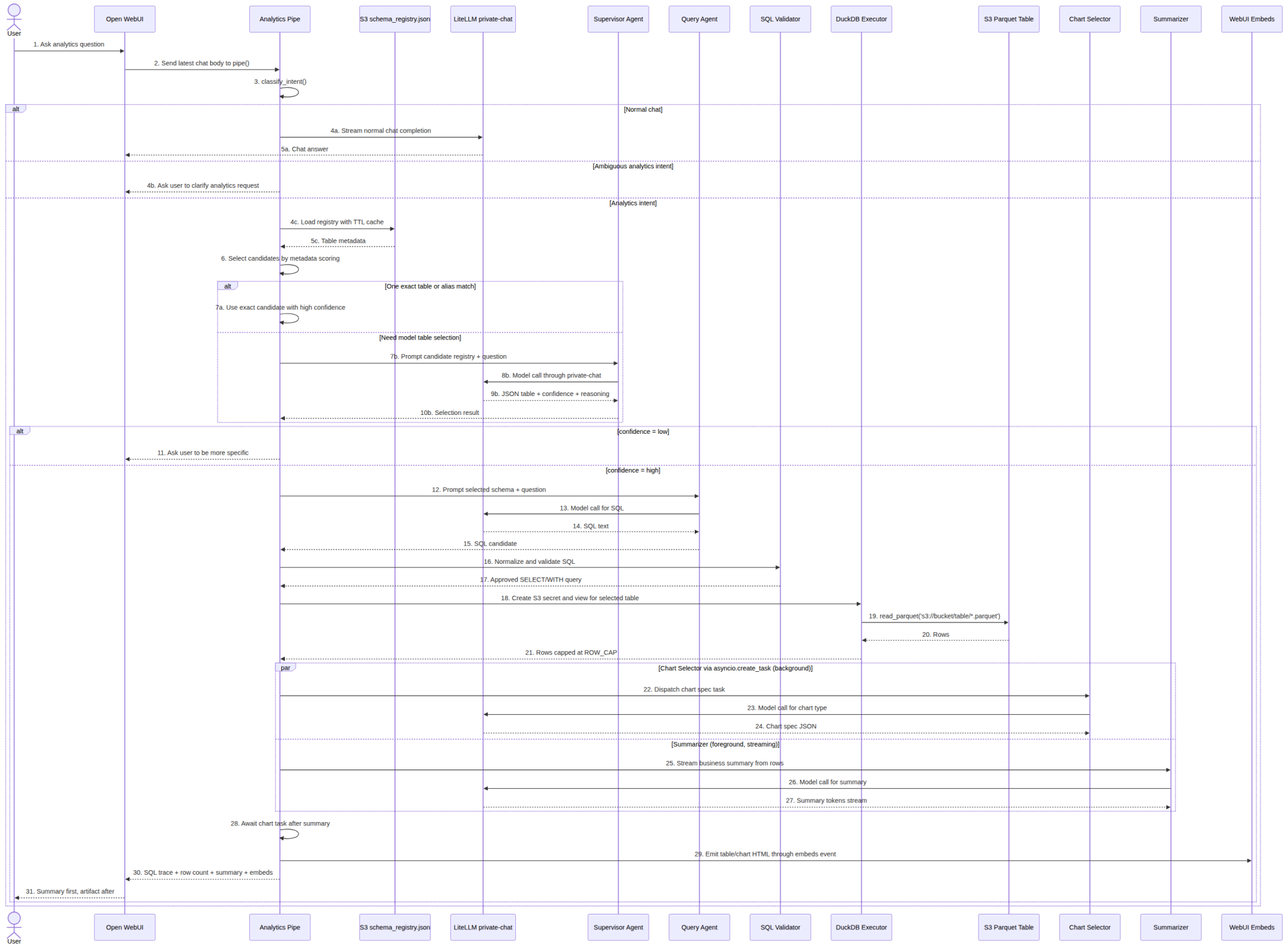
Agent and Guardrail Responsibilities

Component	File / function	Role
Intent Classifier	<code>classify_intent()</code>	Classifies the latest user message as normal chat, ambiguous data request, or analytics request.
	<code>_select_table_candidates()</code>	

Component	File / function	Role
Candidate Selector		Scores likely tables using table name, aliases, columns, dimensions, measures, descriptions, and examples.
Supervisor Agent	<code>_run_supervisor()</code> / <code>analytics_agent/agents/supervisor.py</code>	Selects exactly one table and returns <code>confidence: high low</code> . On low confidence, also suggests up to 3 alternative table names from the candidate list.
Query Agent	<code>_run_query()</code> / <code>analytics_agent/agents/query.py</code>	Generates one SQL statement for the selected table schema. On SQL validation or DuckDB failure, feeds the error back to the model for one self-repair retry before propagating the error.
SQL Validator	<code>_validate_sql()</code> / <code>validate_sql()</code>	Rejects unsafe SQL before DuckDB runs it.
DuckDB Executor	<code>_run_query()</code> / <code>_execute_duckdb()</code>	Creates the DuckDB view over S3 Parquet and returns capped rows.
Chart Selector	<code>_run_chart_spec()</code>	Selects chart or table output when the user asks for visual presentation.
Summarizer	<code>_stream_summary()</code> / <code>analytics_agent/agents/summarize.py</code>	Streams a summary from returned rows, not from raw files.

Confidence is a supervisor routing decision. If confidence is `low`, the pipeline stops before SQL. When candidates are available it suggests up to 3 alternative table names from the candidate list; otherwise it asks the user to clarify. Confidence is not calculated from row count, latency, or numeric SQL metrics.

3. Agents Query DB Flow

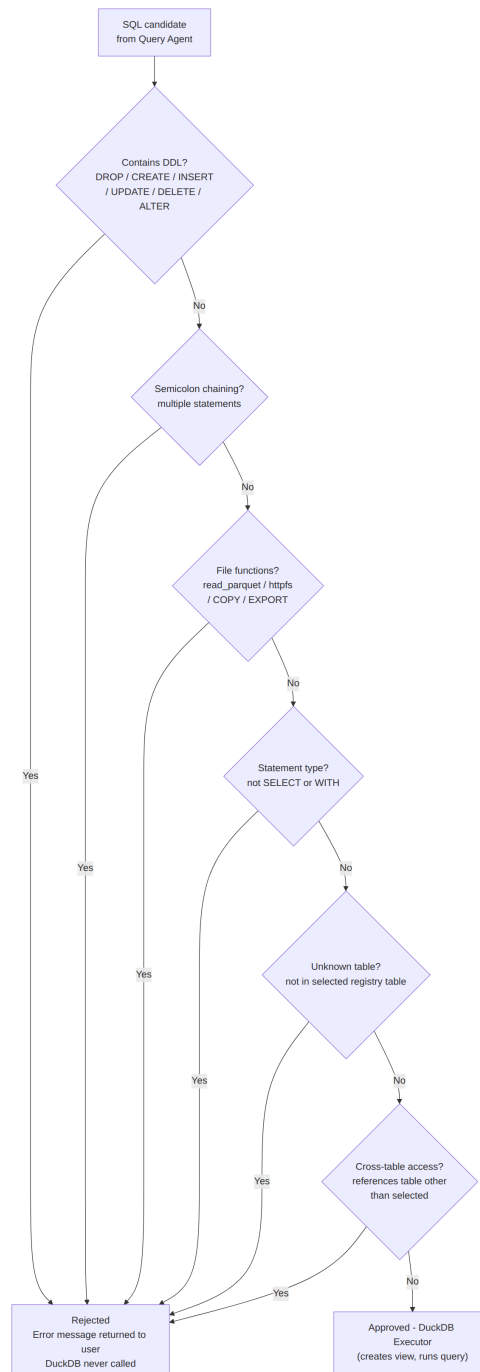


Key Processing Details

- 1. Intent routing:** `classify_intent()` scores the message for NYC Taxi domain signals and an analytics verb. Ambiguous messages (data topic but no clear analytics intent) get a clarification prompt; non-analytics messages pass through to `_llm_chat()` as normal chat.
- 2. Registry cache:** `_load_registry()` returns the in-memory `_registry_cache` on TTL hit. On miss it calls `_fetch_registry_from_s3()`, which manually exchanges the pod's IRSA web identity token with STS and fetches `schema_registry.json` over a Signature V4-signed HTTPS request. On S3 error it returns stale cache rather than failing; first-call failure raises and the pipeline returns an error message.
- 3. Candidate selection and supervisor scope:** `_select_table_candidates()` scores the registry by name, aliases, columns, dimensions, measures, and examples, returning up to 8 candidates. The Supervisor receives only this scored subset via `_candidate_registry()` — not the full registry. If the lexical scorer produces zero candidates, `_stream_analytics()` falls back to passing the full registry; a table scoring zero lexically is not recoverable at the candidate stage otherwise. `_run_supervisor()` forces `confidence=low` if the model returns a table name absent from the provided registry.
- 4. SQL generation with self-repair:** `_run_query()` calls the Query Agent up to twice (`for attempt in range(2)`). If `_validate_sql()` raises `SQLValidationError` or DuckDB raises `duckdb.Error` on attempt 0, `_retry_prompt()` appends the error to the conversation and retries once. The second failure propagates to the caller.
- 5. SQL validation:** `_validate_sql()` is a deterministic function — no model call. It allows only `SELECT` or `WITH` and rejects DDL keywords, semicolons, file functions (`read_parquet()`, `httpfs`, `COPY`), table names not in the registry, and references to any table other than the selected one. DuckDB is never called if validation fails.
- 6. DuckDB execution:** `_build_duckdb_conn()` opens an in-memory DuckDB connection with `memory_limit=512MB` and `extension_directory=/tmp/duckdb-extensions`, runs `INSTALL httpfs; LOAD httpfs` (cached after first download), creates an S3 secret via `_create_s3_secret()`, then creates a view `CREATE VIEW <table> AS SELECT * FROM read_parquet('s3:// ...')`. SQL runs in a `ThreadPoolExecutor(max_workers=1)` with a 30-second `future.result(timeout=...)` guard; concurrent analytics requests queue behind each other.
- 7. Parallel chart and summary:** After DuckDB returns rows, `asyncio.create_task()` dispatches the Chart Selector LLM call as a non-blocking background task. The Summarizer starts streaming immediately in the foreground. The pipeline awaits the chart task only after the summary stream completes. If the chart task raises, the pipeline silently falls back to a table artifact.

SQL Validator — Syntax Guardrail

The SQL validator is a deterministic layer that runs **before DuckDB executes anything**. It checks syntax and structural safety only. Result-level validation (e.g. checking whether returned data is semantically correct) is not yet implemented and is planned as a chain-of-thought eval step in a future iteration.



Current scope: syntax and structural checks only. Data-level validation (checking whether query results are semantically meaningful) is not yet implemented — planned as a chain-of-thought eval step.

Error Handling Paths

Each pipeline stage has its own failure behavior. The table below documents what the pipeline returns at each point — no stage silently swallows errors that should surface to the user, except the Chart Selector which degrades to a table artifact.

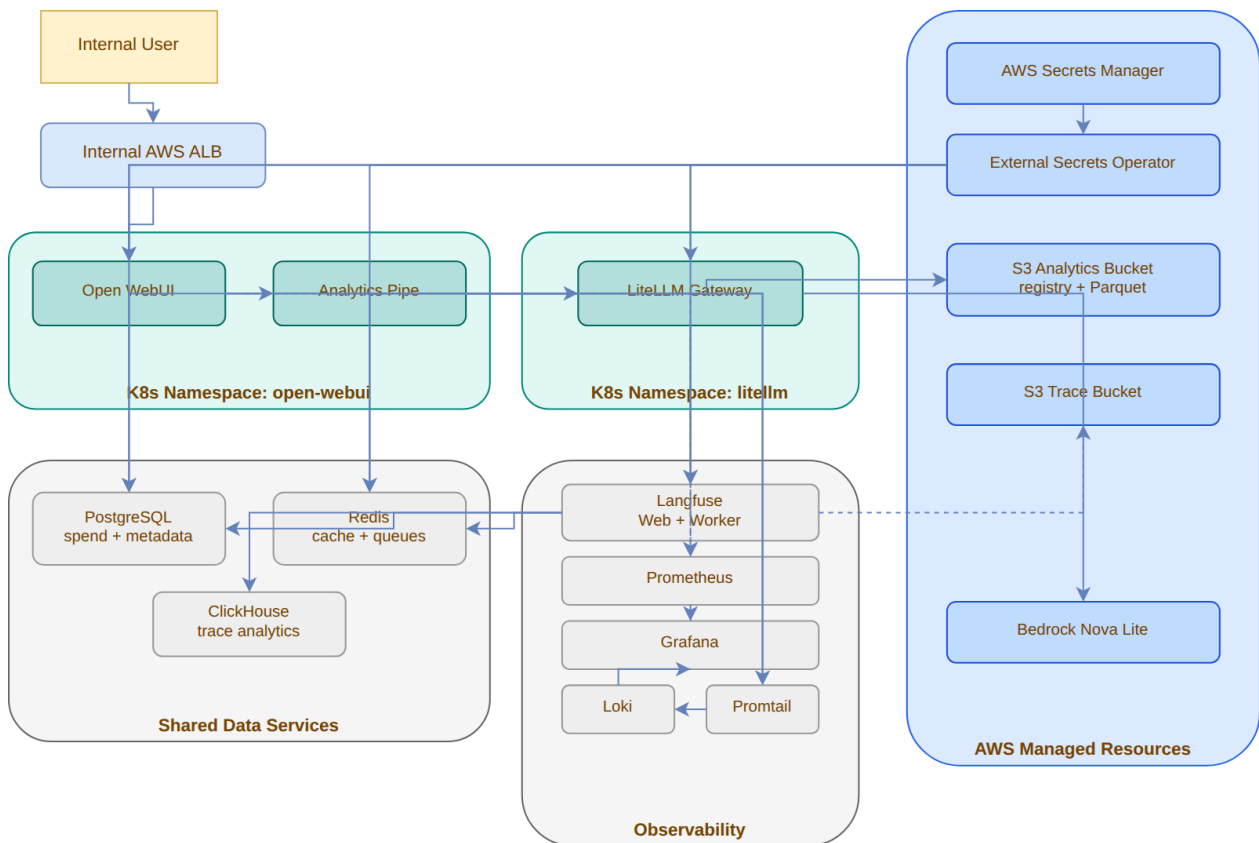
Failure point	Function	Pipeline behavior
Registry fetch fails (no stale cache)	<code>_load_registry()</code> / <code>_fetch_registry_from_s3()</code>	Yields <code>> **Error:** Could not load schema registry - ...</code> , pipeline returns immediately. No SQL, no DuckDB.
No candidates from lexical scorer	<code>_select_table_candidates()</code> → empty	Supervisor receives the full registry as fallback (<code>if prompt_candidates else registry</code>). Zero-scoring tables remain eligible.
Supervisor returns table not in registry	<code>_run_supervisor()</code> line 1122	Forces <code>confidence=low</code> with empty table name and a "not listed in registry" reasoning string. Pipeline stops with clarification prompt.
SQL Validator rejects twice	<code>_validate_sql()</code> + <code>_retry_prompt()</code>	Second <code>SQLValidationError</code> propagates out of <code>_run_query()</code> . Pipeline yields <code>> **Error:** ...</code> to user.
DuckDB times out (30 s)	<code>_execute_sql()</code> via <code>ThreadPoolExecutor</code>	<code>FuturesTimeoutError</code> converted to <code>TimeoutError("DuckDB query exceeded 30s")</code> , returned as error.
Chart Selector fails	<code>_run_chart_spec()</code>	Exception caught at <code>except Exception: traceback.print_exc()</code> . Falls back to table artifact silently; <code>artifact_note</code> shown only if the table HTML also fails.
Summarizer stream fails	<code>_stream_summary()</code>	Caught in caller; falls back to synchronous <code>_llm_chat()</code> call. Summary is still returned, not dropped.

Approximate Latency Budget

Rough estimates per pipeline step at ap-southeast-1 under light traffic. LLM latency is the dominant cost.

Pipeline step	Estimated time	Notes
Registry load (TTL hit)	< 1 ms	Module-level dict lookup
Registry load (cold, S3 fetch)	200–600 ms	STS exchange + S3 HTTP GET + Signature V4
Intent classification	< 5 ms	Deterministic keyword scoring, no model call
Lexical candidate scoring	< 5 ms	In-process registry scan
Supervisor LLM call	800–2 000 ms	Single model round-trip via LiteLLM
Query Agent LLM call	800–2 000 ms	Per attempt; up to ×2 on self-repair
DuckDB execution	200–3 000 ms	Varies with Parquet size and S3 I/O
Chart Selector LLM call	600–1 500 ms	Runs concurrently with Summarizer via <code>asyncio.create_task()</code>
Summarizer streaming	1 000–3 000 ms	Token-by-token via LiteLLM
End-to-end (happy path, cached registry)	~3–8 s	DuckDB and LLM calls dominate

4. Service Connection Flow

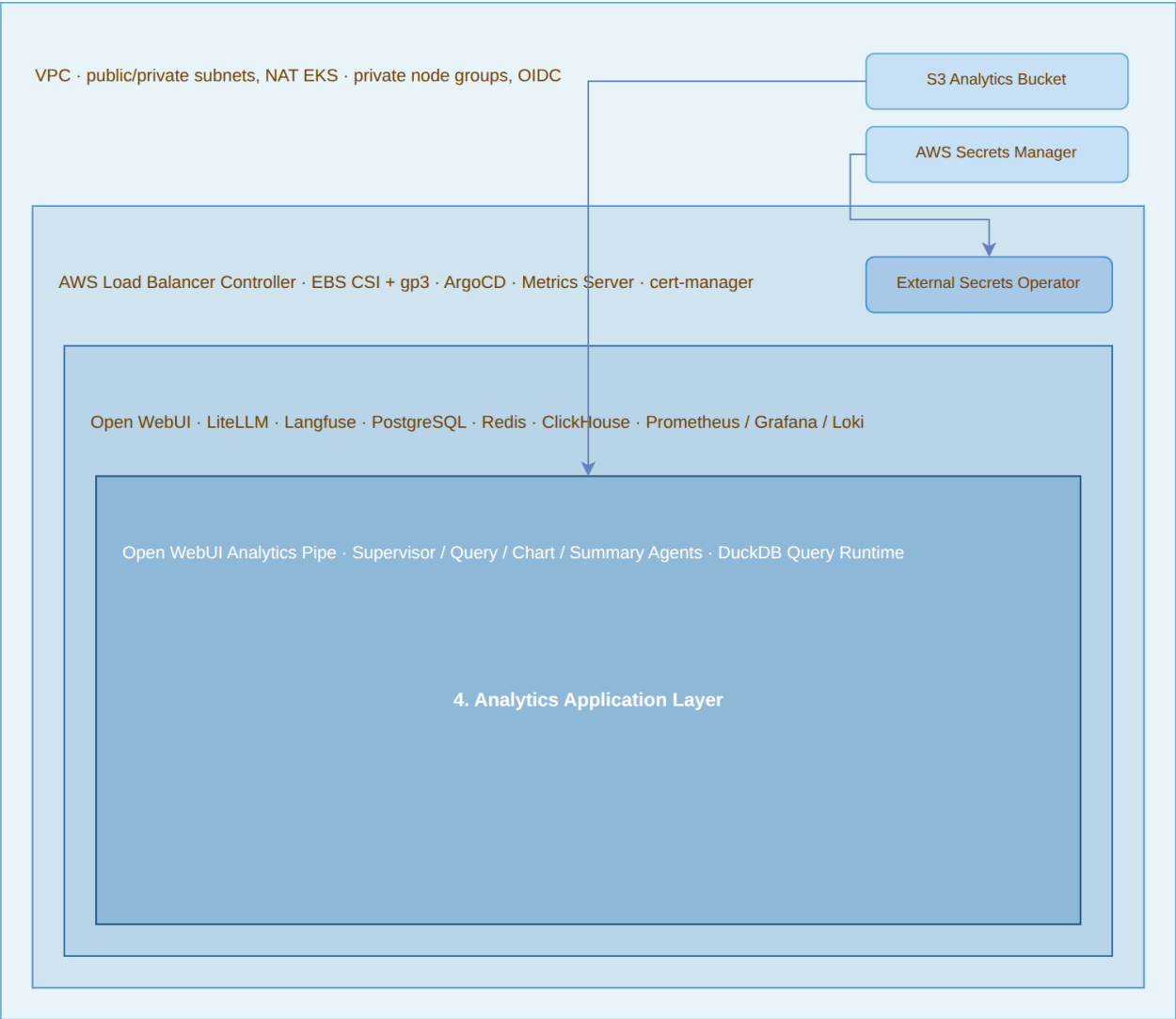


Service Ownership in GitOps

Service	Repo location
Open WebUI	<code>argocd/apps/open-webui.yaml</code> , <code>argocd/helm-values/open-webui-values.yaml</code>
LiteLLM	<code>argocd/apps/litellm.yaml</code> , <code>argocd/helm-values/litellm-values.yaml</code>
Langfuse	<code>argocd/apps/langfuse.yaml</code> , <code>argocd/helm-values/langfuse-values.yaml</code>
PostgreSQL	<code>argocd/apps/postgresql.yaml</code> , <code>argocd/manifests/postgresql.yaml</code>
Redis	<code>argocd/apps/redis.yaml</code> , <code>argocd/helm-values/redis-values.yaml</code>
Loki / Promtail	<code>argocd/apps/loki.yaml</code> , <code>argocd/apps/promtail.yaml</code>
Prometheus / Grafana	<code>argocd/apps/kube-prometheus-stack.yaml</code> , <code>argocd/monitoring/</code>

Service	Repo location
NetworkPolicies	<code>argocd/apps/network-policies.yaml</code> , <code>argocd/monitoring/network-policies.yaml</code>
ExternalSecrets	<code>argocd/apps/open-webui-secrets.yaml</code> , <code>argocd/external-secrets/</code>

5. Platform Layers

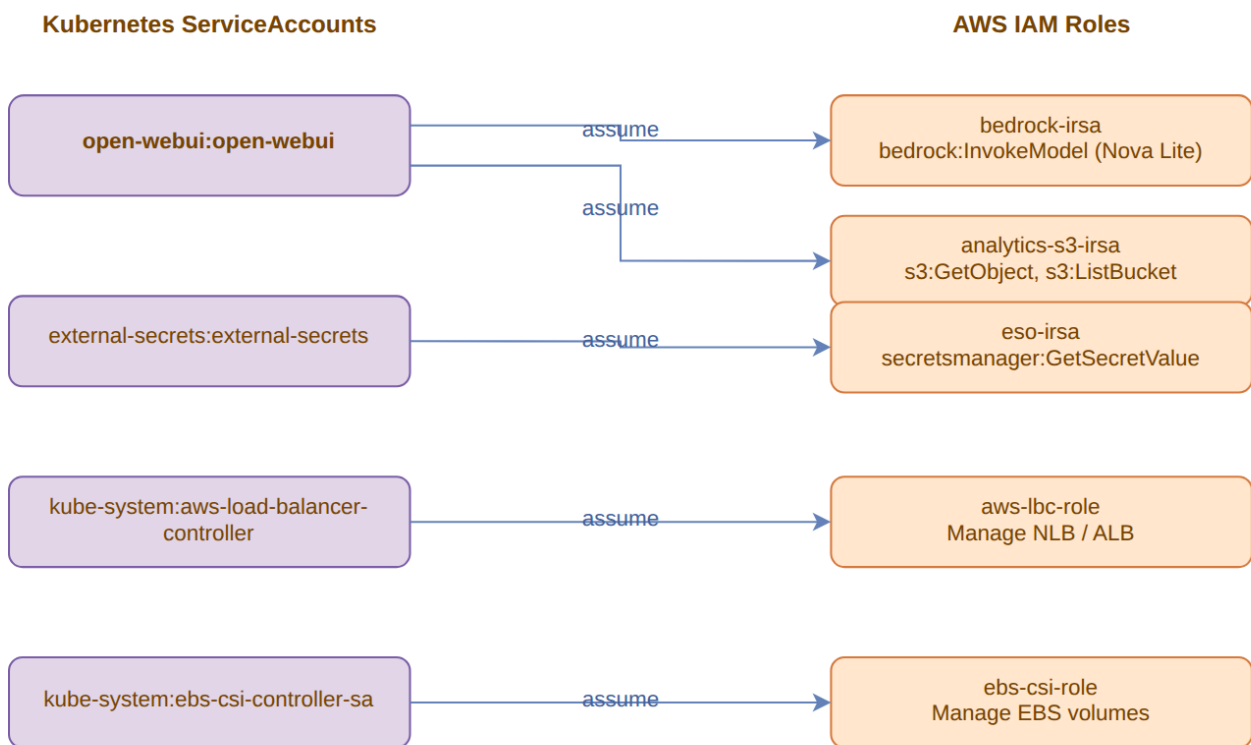


Layer	Responsibility
<code>terraform/vpc</code>	VPC, public/private subnets, routing, NAT, subnet tags.
<code>terraform/eks</code>	EKS control plane, private node groups, OIDC, cluster add-ons.

Layer	Responsibility
<code>terraform/bootstrap</code>	ArgoCD, ALB Controller, EBS CSI, ESO, metrics-server, cert-manager, storage class, Bedrock IRSA, analytics S3 IRSA.
<code>terraform/app</code>	Application-level secrets and supporting resources.
<code>argocd/root-app.yaml</code>	App-of-apps entrypoint.
<code>argocd/apps/</code>	Child apps for WebUI, LiteLLM, Langfuse, PostgreSQL, Redis, Loki, Promtail, monitoring, network policies, RBAC, traffic simulator.

IAM Roles for Service Accounts (IRSA)

To follow the least-privilege principle, no application uses the Node Instance Profile. Each workload that needs AWS API access gets its own IAM Role bound to a specific Kubernetes ServiceAccount via the cluster's OIDC provider.



IRSA Role	Kubernetes ServiceAccount	AWS Permission
<code>bedrock-irsa</code>	<code>open-webui:open-webui</code>	<code>bedrock:InvokeModel</code> on Bedrock Nova Lite
	<code>open-webui:open-webui</code>	

IRSA Role	Kubernetes ServiceAccount	AWS Permission
<code>analytics-s3-irsa</code>		<code>s3:GetObject</code> , <code>s3:ListBucket</code> on analytics bucket
<code>eso-irsa</code>	<code>external-secrets:external-secrets</code>	<code>secretsmanager:GetSecretValue</code> on scoped secrets
<code>aws-lbc-role</code>	<code>kube-system:aws-load-balancer-controller</code>	Manage NLB / ALB resources
<code>ebs-csi-role</code>	<code>kube-system:ebs-csi-controller-sa</code>	Create, attach, detach, delete EBS volumes

S3 Access and IRSA Credential Workaround

The Open WebUI pod uses IRSA (`analytics-s3-irsa`) to access S3, but **both** S3 code paths — the registry HTTP fetch (`_fetch_registry_from_s3`) and the DuckDB S3 secret (`_create_s3_secret`) — implement a manual STS exchange instead of relying on the AWS SDK credential chain or DuckDB's built-in `PROVIDER CREDENTIAL_CHAIN`.

Why: DuckDB 1.2.2's `CREDENTIAL_CHAIN` provider does not resolve the `AWS_WEB_IDENTITY_TOKEN_FILE` environment variable into usable S3 credentials inside the Open WebUI pod. The variable is set by EKS IRSA, but DuckDB's credential walk does not perform the `AssumeRoleWithWebIdentity` STS exchange needed to convert it to temporary AWS credentials.

What the code does instead: When `AWS_ROLE_ARN` and `AWS_WEB_IDENTITY_TOKEN_FILE` are present: 1. Reads the web identity token from the file path in `AWS_WEB_IDENTITY_TOKEN_FILE`. 2. Calls `STS AssumeRoleWithWebIdentity` directly via `urllib.request` to obtain temporary `AccessKeyId`, `SecretAccessKey`, and `SessionToken`. 3. **Registry fetch**

(`_fetch_registry_from_s3`): constructs a manually signed `AWS4-HMAC-SHA256` GET request and fetches `schema_registry.json` over HTTPS using those temporary credentials. 4.

DuckDB S3 secret (`_create_s3_secret`): passes the temporary credentials into `CREATE OR REPLACE SECRET _s3 (TYPE S3, PROVIDER CONFIG, KEY_ID ..., SECRET ..., SESSION_TOKEN ...)` so DuckDB can read S3 Parquet files.

If `AWS_ROLE_ARN` is absent (local development), the registry fetch falls back to an unsigned request and DuckDB falls back to `PROVIDER CREDENTIAL_CHAIN`.

Cluster Bootstrap Order

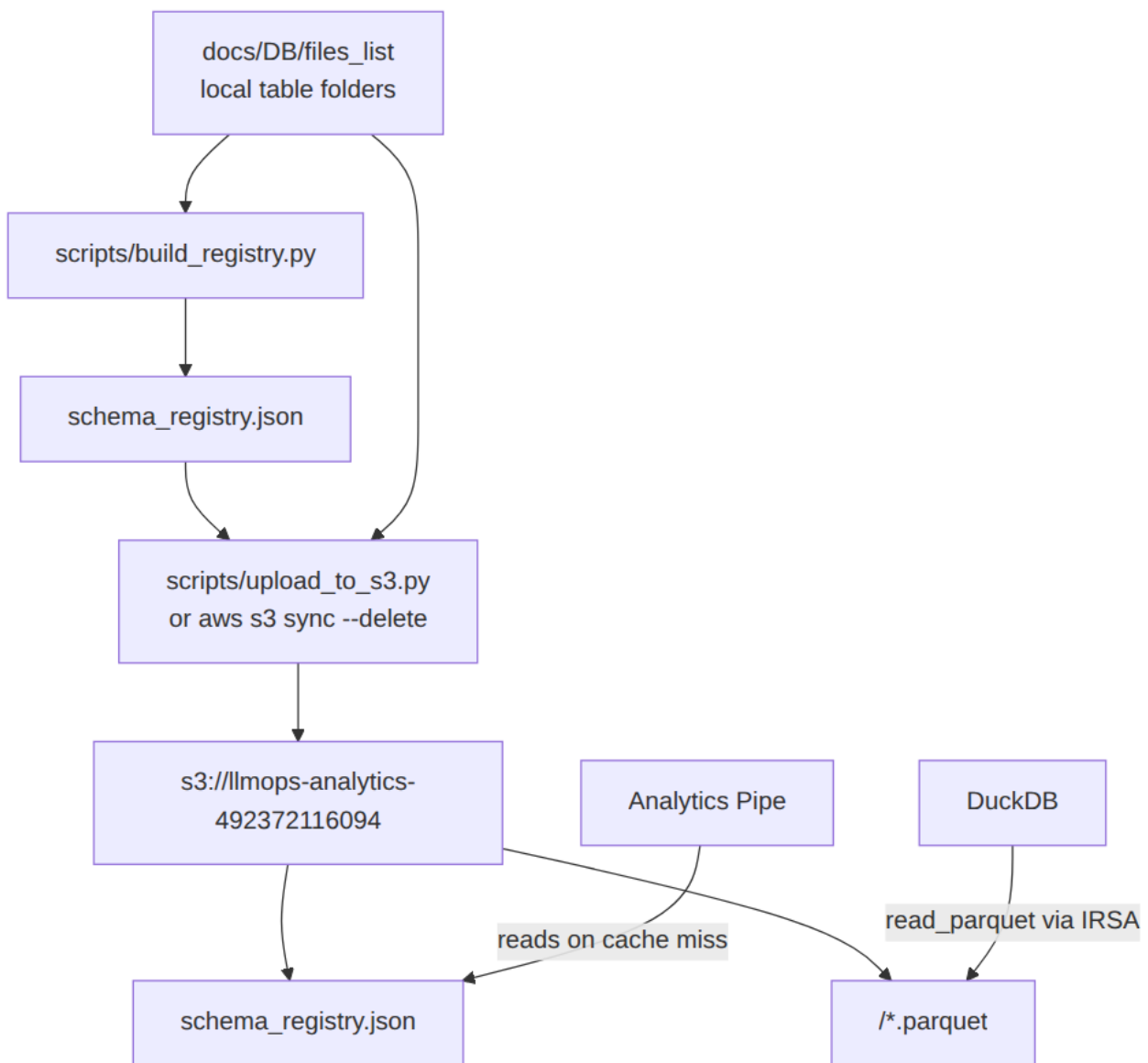
Bringing up the platform from a fresh AWS account follows five sequential steps. Each step must complete before the next begins.

1. `terraform apply` — **vpc + eks layers**: VPC, subnets, NAT, EKS control plane, node groups, OIDC provider.
 2. `terraform apply` — **bootstrap layer**: ArgoCD, ALB Controller, EBS CSI, ESO, metrics-server, cert-manager installed on EKS.
 3. `terraform apply` — **app layer**: S3 analytics bucket, AWS Secrets Manager shells, IRSA roles, K8s namespaces created.
 4. **Manual secret population**: `aws secretsmanager put-secret-value` for API keys, DB passwords, and tokens — one-time step per environment.
 5. `kubectl apply -f argocd/root-app.yaml`: ArgoCD takes over and reconciles all child apps (WebUI, LiteLLM, Langfuse, PostgreSQL, Redis, observability stack, network policies).
-

6. Analytics Data Layer

The current analytics DB is **S3-backed Parquet + DuckDB**. The production default bucket in the pipe is `llmops-analytics-492372116094`, region `ap-southeast-1`.

Registry and Parquet Layout



Data freshness — current state: The registry and Parquet files are uploaded **manually** by running `scripts/upload_to_s3.py` or `aws s3 sync --delete`. The Analytics Pipe caches `schema_registry.json` in memory with a 300-second TTL, so schema changes propagate within 5 minutes of the next upload. An automated pipeline for registry generation and promotion (with data-quality gates) is planned but not yet implemented.

DuckDB Connection Configuration

Each analytics query creates a fresh in-memory DuckDB connection via

`_build_duckdb_conn()`:

- `memory_limit = 512MB` — prevents large Parquet scans from exhausting container memory.
- `extension_directory = /tmp/duckdb-extensions` — extensions are downloaded on first use and cached on the container filesystem. Subsequent connections reuse the cached binaries.
- `INSTALL httpfs; LOAD httpfs` — run on every new connection to activate S3 access. The install is a no-op after the first download.
- **S3 secret** — created immediately after `httpfs` load via `_create_s3_secret()` (see IRSA workaround above).
- **View** — `CREATE VIEW <table> AS SELECT * FROM read_parquet('s3://<bucket>/<table>/*.parquet')` so validated SQL references only the bare table name.

Connections are not pooled; each request opens and closes its own connection.

Registry Cache — Concurrency Behavior

`_registry_cache` and `_registry_ts` are module-level globals with no async lock:

```
_registry_cache: dict | None = None # line 774
_registry_ts: float = 0.0           # line 775
```

Under concurrent async requests: - **Cache hit**: multiple coroutines read the same dict simultaneously — safe, the object is not mutated after assignment. - **Cache miss**: multiple coroutines can enter the S3 fetch branch before any one of them writes `_registry_cache`. This triggers multiple simultaneous S3 fetches and STS exchanges; each overwrites `_registry_cache` on completion (last write wins, all fetches return valid data). - **S3 fetch error**: `_load_registry()` returns stale cache rather than raising. On first-call failure (no stale cache to return), it re-raises and the pipeline yields an error message to the user.

Query Safety Model

```
CREATE VIEW selected_table AS
SELECT * FROM read_parquet('s3://bucket/selected_table/*.parquet')
```

- The query agent must not use `read_parquet()`, `httpfs`, `COPY`, or file functions in SQL output.

- SQL output references only the selected table name.
- Deterministic code creates the DuckDB view after SQL validation.
- Returned rows are capped by `ROW_CAP` to protect WebUI rendering and summarization.

7. Open WebUI Application Layer

Integration point	Current behavior
OpenAI-compatible endpoint	Open WebUI points to LiteLLM at <code>http://litellm.litellm.svc.cluster.local:4000/v1</code> .
Pipe entry point	<code>Pipe.pipe()</code> in <code>openwebui/filter_analytics.py</code> .
Routing input	The pipe uses the latest user message to route intent.
Normal chat	Non-analytics messages stream through LiteLLM with <code>model=private-chat</code> .
Analytics status	The pipe emits status events: selecting table, generating SQL, preparing response, writing summary, done.
Text output	Markdown includes selected table, SQL trace, row count, and summary.
Artifacts	The current path sends table/chart HTML through Open WebUI <code>embeds</code> events.
File helper	<code>_persist_html_artifact()</code> exists for file-token iframe rendering, but <code>_stream_analytics()</code> currently emits <code>embeds</code> directly.

Default Analytics Pipe Valves

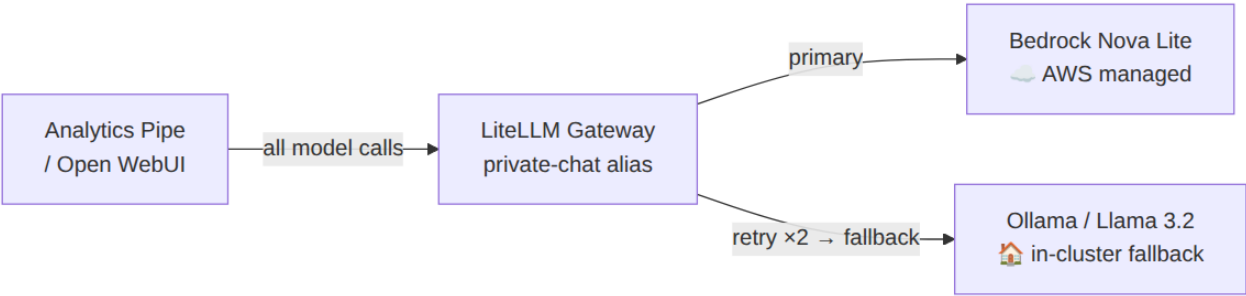
Valve	Default
<code>s3_bucket</code>	<code>llmops-analytics-492372116094</code>
<code>aws_region</code>	<code>ap-southeast-1</code>
<code>litellm_url</code>	<code>http://litellm.litellm.svc.cluster.local:4000/v1/chat/completions</code>
<code>litellm_model</code>	<code>private-chat</code>
<code>registry_ttl</code>	<code>300</code> seconds

Valve	Default
<code>duckdb_timeout</code>	<code>30</code> seconds
<code>row_cap</code>	<code>200</code> rows

Model Alias & Fallback

All agent and chat calls go through LiteLLM using the `private-chat` alias — no service calls a model provider directly. If Bedrock returns errors or times out, LiteLLM retries twice then falls back to the next configured backend.

Alias	Primary Backend	Fallback	Notes
<code>private-chat</code>	Bedrock Nova Lite (ap-southeast-1)	Ollama in-cluster (Llama 3.2)	Used by all agents and normal chat



Ollama is CPU-constrained (RPM limited to 10, timeout 300s). It exists as a last-resort fallback to keep the platform available during Bedrock outages, not as a primary inference backend.

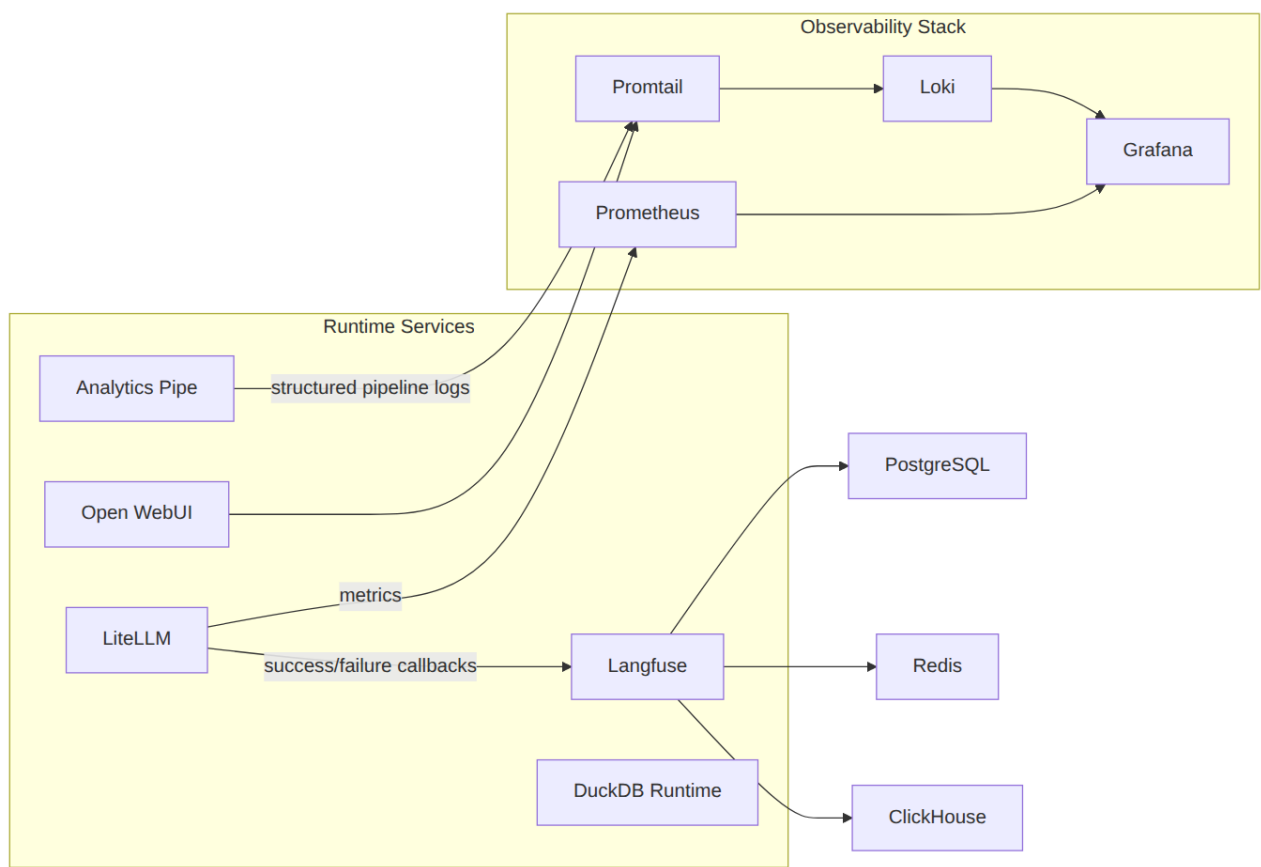
8. Security and Governance

Control	Current design
Central model routing	Open WebUI and agents call LiteLLM instead of direct provider clients.
Current private-chat backend	In EKS, the <code>private-chat</code> alias is Bedrock Nova Lite through LiteLLM.
S3 analytics access	Terraform creates <code>llmops-analytics-*</code> and an Open WebUI IRSA role with read-only S3 access.

Control	Current design
Secret lifecycle	AWS Secrets Manager stores values; External Secrets Operator syncs Kubernetes Secrets.
SQL safety	The query agent can only run validated SQL against the selected registry table.
Data scan control	DuckDB reads only <code>s3://<bucket>/<selected_table>/*.parquet</code> .
Row limits	The pipe caps results and returns at most <code>ROW_CAP</code> rows.
Network posture	EKS nodes run in private subnets; user-facing access goes through internal ALBs.
GitOps control	ArgoCD automated prune/self-heal reconciles cluster resources from Git.

The SQL validator is the most important data-plane guardrail: it rejects DDL, chained statements, file functions, unknown tables, and unauthorized table references before DuckDB executes anything.

9. Observability Flow



Pillar	Tooling	What it covers
LLM traces	LiteLLM callbacks to Langfuse	Model calls, token usage, latency, cost, trace inspection.
Metrics	Prometheus + Grafana	Pod health, gateway latency, DB metrics, Redis metrics, alerts.
Logs	Loki + Promtail	Application logs, analytics pipe errors, request metadata, runtime failures.
Structured analytics logs	<code>analytics_agent/pipeline.py</code>	<code>correlation_id</code> , selected table, confidence, SQL, row counts, capped status, latency, outcome.

Alerting Rules

Key alert rules defined in `argocd/monitoring/alert-rules.yaml`:

Alert	Condition	Severity
<code>LiteLLMHighErrorRate</code>	HTTP 5xx error rate > 5% for 5m	critical
<code>LLMProviderHighTimeoutRate</code>	Timeout rate > 10% for 5m	warning
<code>OpenWebUIDown</code>	<code>up == 0</code> for 2m	critical
<code>LangfuseDown</code>	<code>up == 0</code> for 5m	warning
<code>LiteLLMHighLatency</code>	P95 latency > 3s for 10m	warning

All alerts route to Slack via Alertmanager webhook.

SLO Targets

Service	Target
Open WebUI availability	99.5%
LiteLLM gateway availability	99.9%
P95 LiteLLM latency (excl. provider)	< 3 seconds
LLM request success rate	> 98%
Error alert detection time	< 5 minutes

10. Tradeoffs and Current Limits

Decision / Limit	Reason	Next step when needed
Open WebUI pipe instead of separate dashboard	Analytics stays inside the chat workflow and supports natural follow-up questions.	Split into a dedicated analytics UI only if the chat artifact surface becomes limiting.
S3 Parquet + DuckDB	Cheap, simple, and appropriate for file-oriented analytics.	Move query runtime to a dedicated service or OLAP backend when concurrency grows.
Single-table query policy		

Decision / Limit	Reason	Next step when needed
	Reduces risk from model-generated SQL scanning too broadly or joining incorrectly.	Add semantic joins only when relationships are explicit in the registry.
Separated agent roles	Each prompt has a narrow task, making behavior easier to control and debug.	Add evals per agent before expanding autonomy.
Registry JSON on S3	Easy to publish and consume from the Open WebUI pod.	Add versioning, validation gates, and a promotion workflow.
Best-effort chart selector	Chart failure does not block the summary.	Add deterministic chart heuristics for common result shapes.
Data freshness depends on object sync	Parquet and registry updates are uploaded from generated local data.	Automate <code>aws s3 sync --delete</code> with data-quality checks.
Bedrock Nova Lite as the sole model backend	Cost-efficient for short analytics queries; SQL generation and summarization do not require a large reasoning model at current scale.	Switch to a stronger model if SQL quality or summary accuracy proves insufficient in evaluation.
<code>ThreadPoolExecutor(max_workers=1)</code> for DuckDB	Limits DuckDB to one concurrent SQL execution; additional analytics requests queue behind it rather than running in parallel.	Move DuckDB execution to a dedicated worker pool or a separate query service when concurrency grows.
Registry cache (<code>_registry_cache</code>) with no async lock	Concurrent cache misses can trigger multiple simultaneous S3 fetches and STS exchanges; stale data is returned on fetch failure instead of erroring.	Add an <code>asyncio.Lock</code> around the cache refresh path when simultaneous S3 round-trips become a concern.